

DoConnect AI

Intelligent Discussion & Knowledge Collaboration Platform

PROJECT REPORT

Great Learning — AI Capstone Project

Project Title	DoConnect AI
Submitted By	Poojitha Bhukya
Program	Java FSD React
Mentor / Trainer	Javeed
Organization	Great Learning
Submission Date	18-06-2026

Abstract

DoConnect AI is an intelligent discussion and knowledge collaboration platform designed to enable users to ask questions, share answers, collaborate through real-time communication, and leverage artificial intelligence for enhanced knowledge discovery. The platform integrates Google Gemini AI for automated answer generation, tag recommendations, sentiment analysis, and content moderation. It provides role-based access control for Users, Moderators, and Administrators. The system also includes analytics dashboards, content moderation workflows, and WebSocket-based group chat functionality. The project aims to improve knowledge sharing, collaboration, and community management in online discussion environments.

1. Problem Statement

In today's digital era, online discussion platforms play a vital role in knowledge sharing and collaborative learning. Traditional discussion forums allow users to post questions and answers; however, they often lack intelligent assistance, real-time communication capabilities, effective content moderation mechanisms, and analytical insights. As the volume of user-generated content increases, it becomes difficult for users to discover relevant information quickly and for administrators to maintain content quality and community standards.

Many existing platforms require users to manually search through large amounts of information to find suitable answers. Additionally, inappropriate, irrelevant, or spam content can negatively impact the overall user experience. Community managers and moderators face challenges in identifying problematic content and monitoring user engagement efficiently. Furthermore, the absence of real-time communication limits collaboration and knowledge exchange among users.

To address these challenges, there is a need for an intelligent discussion platform that combines traditional question-and-answer functionality with Artificial Intelligence, real-time communication, content moderation, analytics, and role-based administration. The proposed system, DoConnect AI, is designed to provide users with an enhanced knowledge-sharing environment where AI assists in content generation, moderation, and discovery while administrators and moderators can effectively manage the platform and its users.

1. Introduction

1.1 Description

DoConnect AI is a full-stack, AI-integrated discussion and knowledge collaboration platform conceptualized as a hybrid of StackOverflow, Discord, and modern generative AI capabilities. The platform enables users to post technical questions, receive instantly generated AI answers, engage in real-time discussions through WebSocket-based chat rooms, and benefit from intelligent content moderation and analytics.

The system follows an enterprise-grade layered architecture and supports strict role-based access control across three user roles — User, Moderator, and Admin — ensuring secure and structured usage across the platform.

1.2 Project Overview

This capstone project simulates a real-world, production-style Q&A and collaboration platform. Unlike a traditional Q&A system, DoConnect AI embeds Google's Gemini AI directly into the core workflow: every question automatically receives an AI-generated answer, every post is screened for toxic content before publishing, smart tags are suggested using AI, and platform-wide sentiment is continuously analyzed for the admin dashboard.

Authentication is handled through JWT (JSON Web Tokens), ensuring stateless, secure communication between the React frontend and the Spring Boot backend. Real-time features are powered by WebSocket (STOMP over SockJS), allowing instantaneous message delivery across chat rooms without page refreshes.

1.3 Objectives

- Enable structured, AI-assisted knowledge sharing between developers
- Provide secure, token-based authentication and role-based authorization
- Automate content moderation using AI to maintain platform quality
- Support real-time, multi-room discussion through WebSocket communication
- Deliver actionable analytics through an administrative dashboard
- Demonstrate enterprise design patterns and clean layered architecture

2. Technologies Used

The application is built using a modern, industry-standard technology stack chosen for scalability, security, and developer productivity.

2.1 Frontend

Technology	Purpose
React 18 (Hooks)	Component-based UI development
Redux Toolkit	Centralized global state management
React Router v6	Client-side routing and navigation
Axios	REST API communication with JWT interceptors
Bootstrap 5	Responsive layout and base styling
Custom Theme (CSS)	Branded design system across all pages
STOMP.js + SockJS	Real-time WebSocket chat communication
Recharts	Analytics charts on the admin dashboard

2.2 Backend

Technology	Purpose
Spring Boot 3.2	Core backend application framework
Spring Security	Authentication and authorization
JWT (jjwt)	Stateless, token-based session security
Spring Data JPA / Hibernate	Object-relational database mapping
Spring WebSocket	Real-time messaging infrastructure
Lombok	Boilerplate reduction in Java classes
springdoc-openapi (Swagger)	Auto-generated REST API documentation
JUnit 5 + Mockito	Unit testing of the service layer

2.3 Database

Technology	Purpose
MySQL 8	Persistent relational data storage (7 tables)

2.4 Artificial Intelligence

Technology	Purpose
Google Gemini API	AI answer generation, tag suggestion, moderation, sentiment analysis
OpenAI API	Secondary/backup AI provider (Strategy Pattern)

2.5 Deployment

Technology	Purpose
Docker	Containerized backend packaging
Render	Backend hosting
Netlify	Frontend static hosting
Railway	Managed MySQL database hosting

3. System Architecture

DoConnect AI follows a strict layered architecture pattern that separates concerns across the frontend, backend, and database, ensuring maintainability, testability, and scalability.

3.1 High-Level Architecture Flow

React Frontend → REST Controllers → Service Layer → Repository Layer → MySQL Database

In parallel, the Service Layer communicates with the Google Gemini API for AI-powered features, and a dedicated WebSocket layer (STOMP/SockJS) handles real-time chat independently of the standard REST request/response cycle.

3.2 Backend Layers

Layer	Responsibility
Controller Layer	Exposes 40+ REST endpoints; handles HTTP requests/responses
Service Layer	Implements business logic, validation, and AI orchestration
Repository Layer	Spring Data JPA interfaces for database access
Entity Layer	JPA-mapped classes representing database tables and relationships
DTO Layer	Request/response objects decoupled from entities, with validation annotations
Security Layer	JWT generation/validation, Spring Security filter chain, role-based access
Design Pattern Layer	Factory, Builder, Strategy, Observer, Proxy, and Singleton implementations

3.3 Frontend Layers

Layer	Responsibility
Pages	10 route-level views (Welcome, Login, Home, Chat, Admin Dashboard, etc.)
Components	Reusable UI elements (Navbar, QuestionCard, AnswerCard, route guards)
Redux Store	Global authentication and question state management
Services	Axios-based API clients per domain (auth, question, answer, AI, analytics)
Hooks	Custom WebSocket connection hook for real-time chat
Utils	Centralized client-side validation logic

4. Database Design

The database is designed with seven normalized relational tables, connected through well-defined primary and foreign key relationships to ensure data consistency and eliminate redundancy.

4.1 Entities

Entity	Key Attributes
users	id, username, email, password, role, bio, created_at
questions	id, title, content, votes, views, user_id (FK), created_at
answers	id, content, votes, is_accepted, is_ai_generated, question_id (FK), user_id (FK)
tags	id, name (unique)
question_tags	question_id (FK), tag_id (FK) — join table
chat_messages	id, sender, content, room_id, type, timestamp
flagged_content	id, content_type, content_id, reason, status, flagged_at

4.2 Entity Relationships

1. User ↔ Question (One-to-Many): A user can post multiple questions; each question belongs to exactly one user, enforced via the user_id foreign key.
2. Question ↔ Answer (One-to-Many): A question can have multiple answers; each answer is linked to exactly one question via question_id.
3. User ↔ Answer (One-to-Many): A user can submit multiple answers across different questions.
4. Question ↔ Tag (Many-to-Many): Implemented through the question_tags join table, allowing a question to have multiple tags and a tag to belong to multiple questions.
5. User ↔ ChatMessage (One-to-Many): A user can send multiple chat messages across different rooms.
6. FlaggedContent: References either a question or an answer via content_type and content_id, used by the AI moderation pipeline and reviewed by Moderators/Admins.

4.3 JPA Mapping Strategy

- All collection relationships use FetchType.LAZY to avoid unnecessary eager loading.
- CascadeType.ALL with orphanRemoval is applied on User → Questions/Answers so dependent records are cleaned up automatically.
- @JsonIgnore is used on bidirectional relationship fields to prevent infinite serialization loops in API responses.
- Bean validation annotations (@NotBlank, @Size, @Email, @Min) are applied directly on entity fields in addition to DTOs for defense-in-depth validation.

5. Core Modules

5.1 Authentication & User Roles Module

Handles registration, login, and role assignment (USER, MODERATOR, ADMIN). Issues JWT tokens on successful authentication, which are validated on every subsequent request via a custom JwtAuthFilter.

5.2 AI Question & Answer Module

When a user posts a question, the system asynchronously calls the Gemini API to auto-generate an answer. Users can additionally request AI tag suggestions while composing a question, and can manually trigger AI answer generation from the question detail page.

5.3 Real-Time Discussion & Chat Module

Implemented using Spring WebSocket with STOMP over SockJS. Users can create or join named chat rooms, send messages instantly to all subscribers of that room, and view persisted chat history retrieved from the chat_messages table.

5.4 AI Content Moderation Module

Before a question or answer is saved, its content is passed through Gemini's moderation check. Content flagged as toxic is rejected at submission; borderline content is logged into the flagged_content table for manual review by Moderators or Admins.

5.5 Analytics Dashboard Module

Restricted to Admin users. Aggregates platform-wide statistics — total users, questions, answers, chat messages, flagged content count, trending questions by views/votes, most active users, and an AI-computed sentiment score representing the overall tone of platform content.

5.6 Moderation Module

Accessible to both Moderators and Admins. Displays all pending flagged content with the reason for flagging, and allows reviewers to approve (restore) or reject (permanently remove) the content.

5.7 User Profile Module

Allows users to view and edit their profile information, change their password securely, view their own posted questions and answers, and delete their account.

6. Design Patterns Implemented

Six design patterns were deliberately applied across the backend to solve specific architectural problems, rather than being added superficially.

Pattern	Location	Purpose
Singleton	All @Service / @Repository classes	Spring manages exactly one instance of each service throughout the application lifecycle.
Factory	ResponseFactory.java	Generates consistent success/error/AI response payloads across every controller.
Builder	NotificationBuilder.java	Constructs complex notification objects step-by-step for readability and maintainability.
Strategy	AIStrategy / GeminiStrategy / OpenAIStrategy / AIContext	Allows switching the active AI provider via a single configuration property, with zero code changes elsewhere.
Observer	EventPublisher / NotificationObserver / LoggingObserver	Automatically notifies all interested observers (logging, notifications) whenever a domain event occurs, such as an answer being posted.
Proxy	AIServiceProxy.java	Caches AI-generated answers and tag suggestions in memory, preventing duplicate calls to the Gemini API for repeated questions.

7. REST API Endpoints

The backend exposes over 40 RESTful endpoints, fully documented through Swagger/OpenAPI and accessible at </swagger-ui.html>. All endpoints except authentication require a valid JWT bearer token.

7.1 Authentication

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate and receive JWT token

7.2 Users

Method	Endpoint	Description
GET	/api/users/profile	Get the logged-in user's profile
PUT	/api/users/profile	Update profile details
PUT	/api/users/password	Change account password
GET	/api/users/{id}	Get a specific user's public profile
GET	/api/users/{id}/questions	Get a user's posted questions
GET	/api/users/{id}/answers	Get a user's posted answers
DELETE	/api/users/account	Delete own account

7.3 Questions

Method	Endpoint	Description
GET / POST	/api/questions	List all / create a question
GET / PUT / DELETE	/api/questions/{id}	Get, update, or delete a question
GET	/api/questions/search	Search questions by keyword
GET	/api/questions/trending	Get top trending questions
GET	/api/questions/tag/{tagName}	Get questions filtered by tag
PUT	/api/questions/{id}/vote	Upvote or downvote a question

7.4 Answers

Method	Endpoint	Description
POST	/api/answers	Post an answer to a question
GET	/api/answers/{questionId}	Get all answers for a question
PUT / DELETE	/api/answers/{id}	Update or delete an answer
PUT	/api/answers/{id}/accept	Mark an answer as accepted
PUT	/api/answers/{id}/vote	Upvote or downvote an answer

7.5 AI Features

Method	Endpoint	Description
POST	/api/ai/answer/{questionId}	Generate an AI answer for a question
POST	/api/ai/tags	Get AI-suggested tags
POST	/api/ai/moderate	Check content for policy violations
POST	/api/ai/sentiment	Analyze sentiment of given text

7.6 Chat

Method	Endpoint	Description
WS	/ws	WebSocket (STOMP/SockJS) connection endpoint
GET	/api/chat/{roomId}/history	Get persisted chat history for a room

7.7 Analytics (Admin)

Method	Endpoint	Description
GET	/api/analytics/dashboard	Get aggregated dashboard statistics
GET	/api/analytics/trending	Get trending topics data
GET	/api/analytics/active-users	Get most active users

7.8 Moderation (Admin / Moderator)

Method	Endpoint	Description
GET	/api/moderation/pending	Get all pending flagged content
PUT	/api/moderation/{id}/approve	Approve flagged content

Method	Endpoint	Description
PUT	/api/moderation/{id}/reject	Reject and remove flagged content
POST	/api/moderation/flag	Manually flag a piece of content

7.9 Admin

Method	Endpoint	Description
GET	/api/admin/users	Get all registered users
PUT	/api/admin/users/{id}/role	Update a user's role
DELETE	/api/admin/users/{id}	Delete a user account

8. Security Implementation

8.1 JWT-Based Authentication

On successful login, the backend issues a signed JWT containing the user's email as the subject claim. Every protected request must include this token in the Authorization header as a Bearer token. A custom `JwtAuthFilter` intercepts each request, validates the token's signature and expiry, and populates the Spring Security context accordingly.

8.2 Role-Based Access Control

Role	Access Level
USER	Post/answer questions, vote, chat, manage own profile
MODERATOR	All USER permissions plus access to the content moderation queue
ADMIN	All MODERATOR permissions plus full analytics dashboard and user management

8.3 Password Security

Passwords are never stored in plain text. `BCryptPasswordEncoder` is used to hash passwords before persistence, and password change requests require verification of the current password before accepting a new one.

8.4 Global Exception Handling

A centralized `GlobalExceptionHandler` intercepts validation errors, duplicate-resource conflicts, not-found errors, authentication failures, and unexpected exceptions, returning consistent, structured JSON error responses to the frontend.

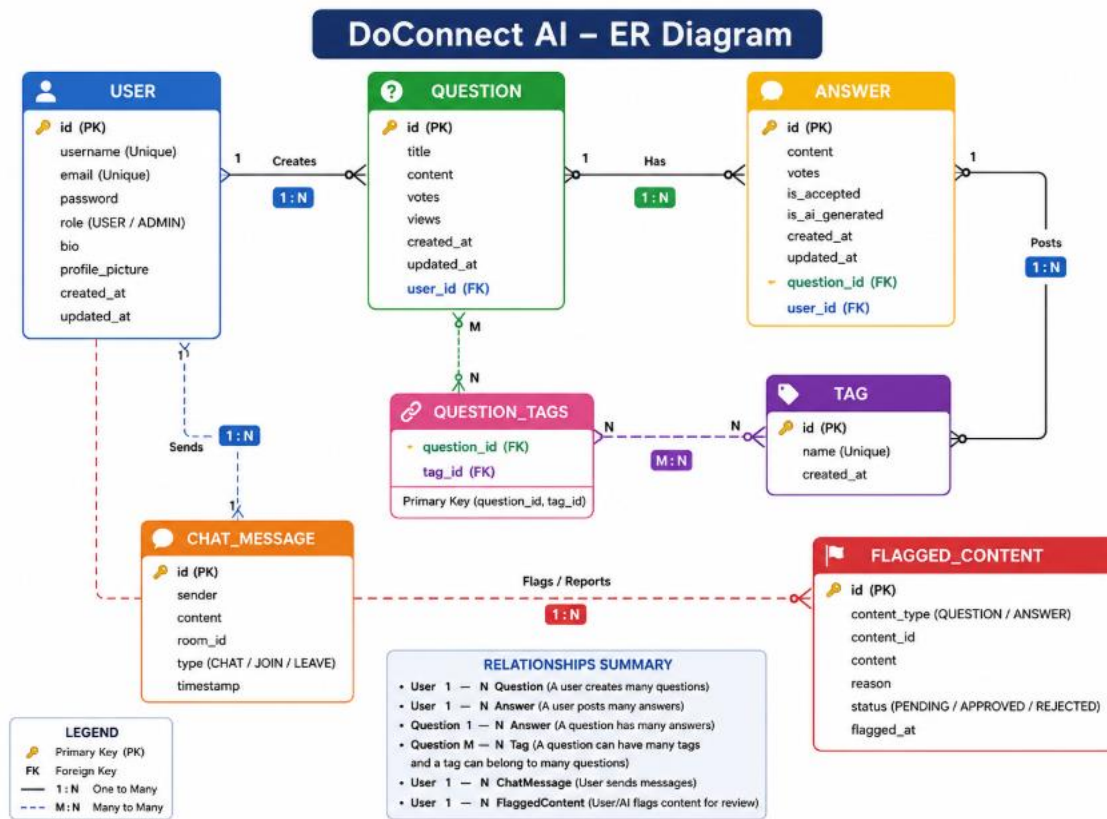
9. Testing

Unit testing was performed exclusively on the service layer, since this is where the core business logic resides. JUnit 5 was used as the testing framework, with Mockito used to mock all repository and external dependencies — ensuring tests run independently of the database or external AI APIs.

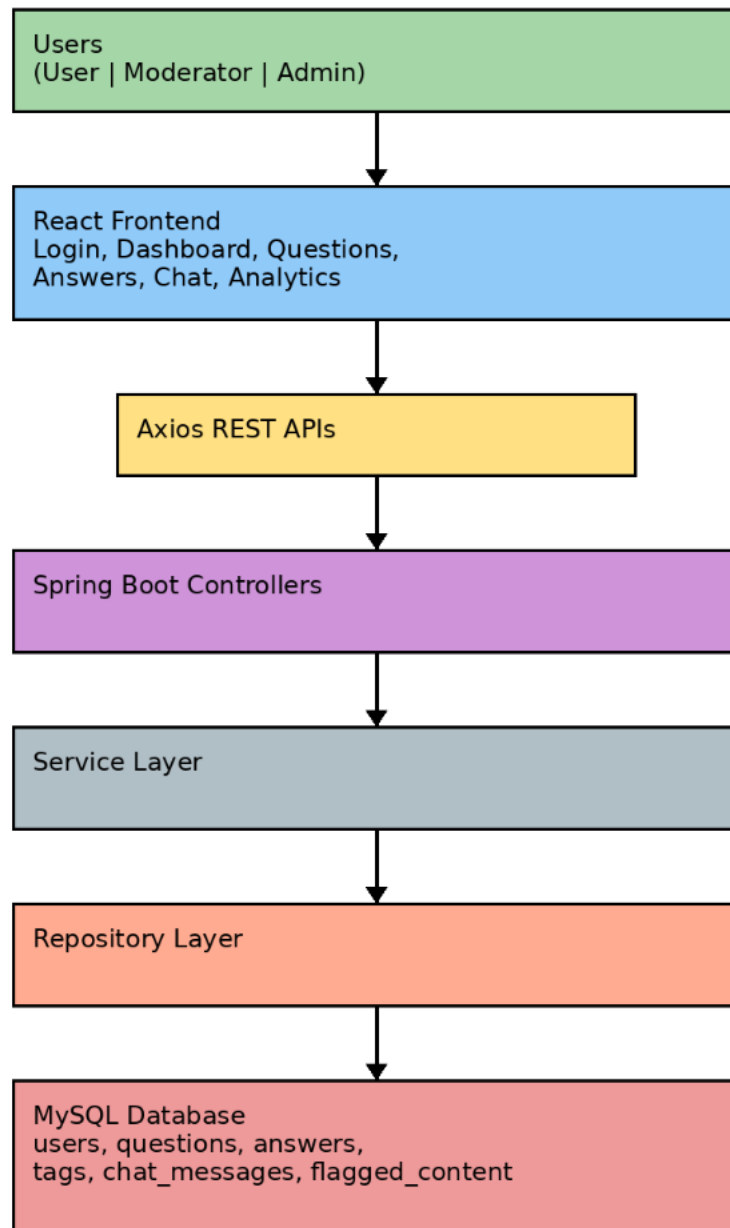
Test Class	Scenarios Covered	Test Count
AuthServiceImplTest	Registration success/conflict, login success/failure	5
UserServiceImplTest	Profile retrieval/update, password change, account deletion	6
QuestionServiceImplTest	CRUD operations, toxic content rejection, search, voting	9
AnswerServiceImplTest	Posting, ownership checks, accept, voting	8
ChatServiceImplTest	Message persistence, room history retrieval	6
AIServiceImplTest	Answer generation, tag suggestion, moderation, sentiment	8

Total: 42 unit tests, all passing, covering happy-path, edge-case, and failure scenarios for every service class.

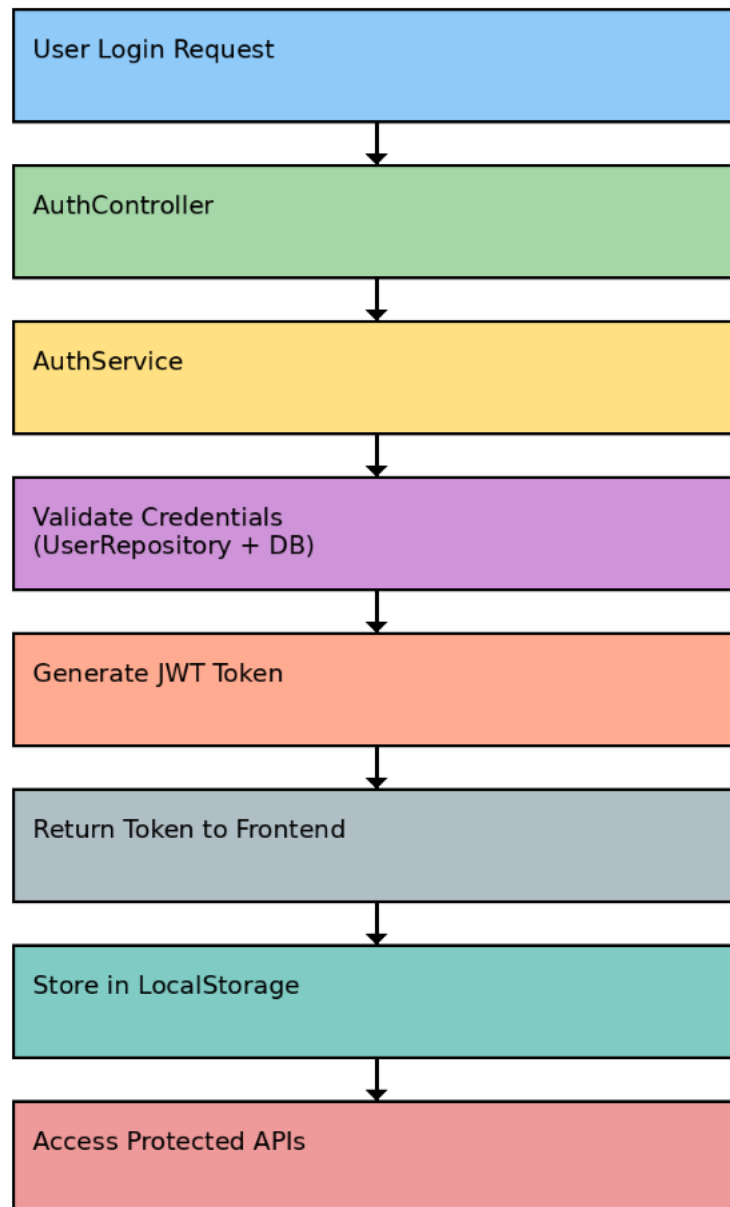
10. ARCHITECTURE AND SYSTEM DIAGRAMS



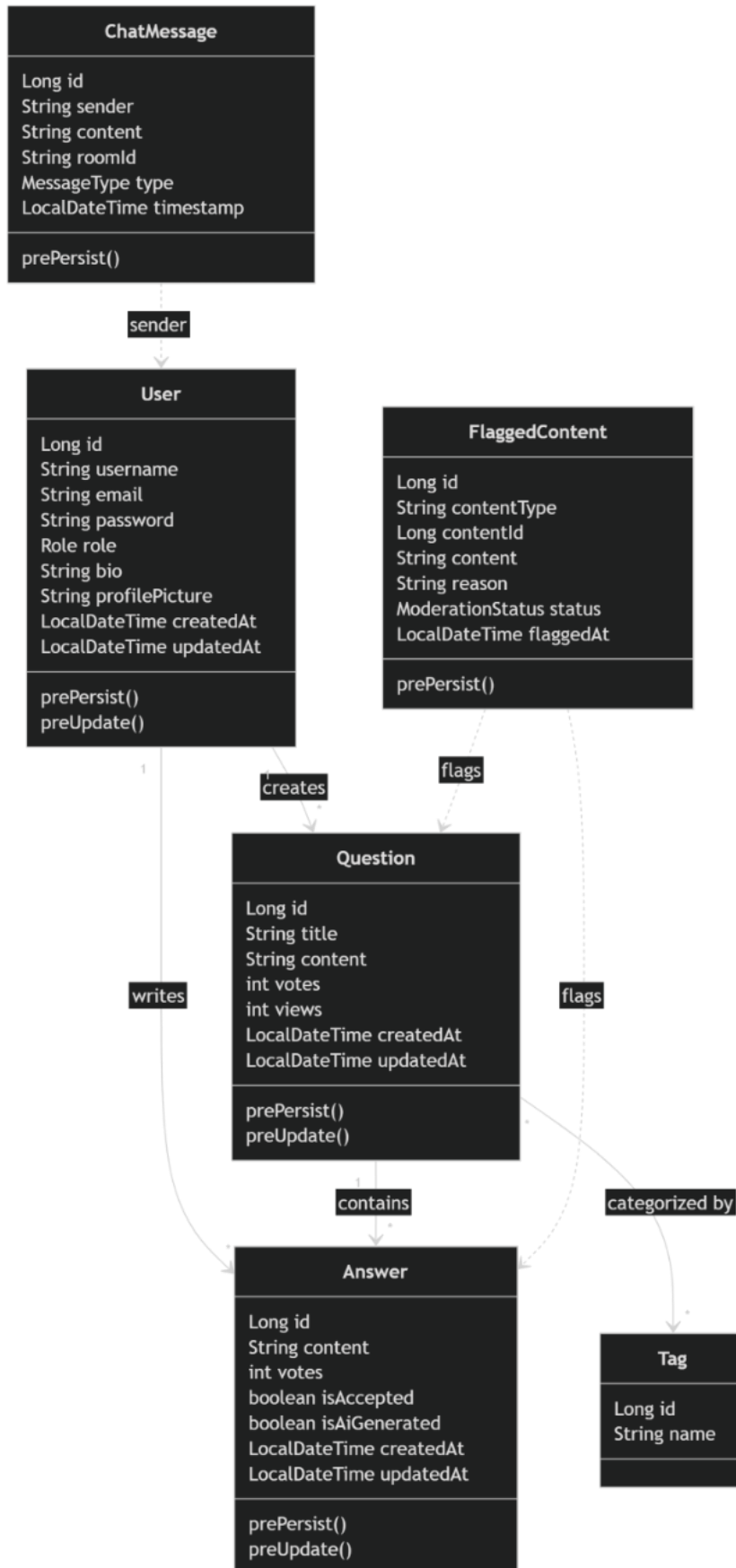
DoConnect AI - System Architecture



DoConnect AI - JWT Authentication Flow



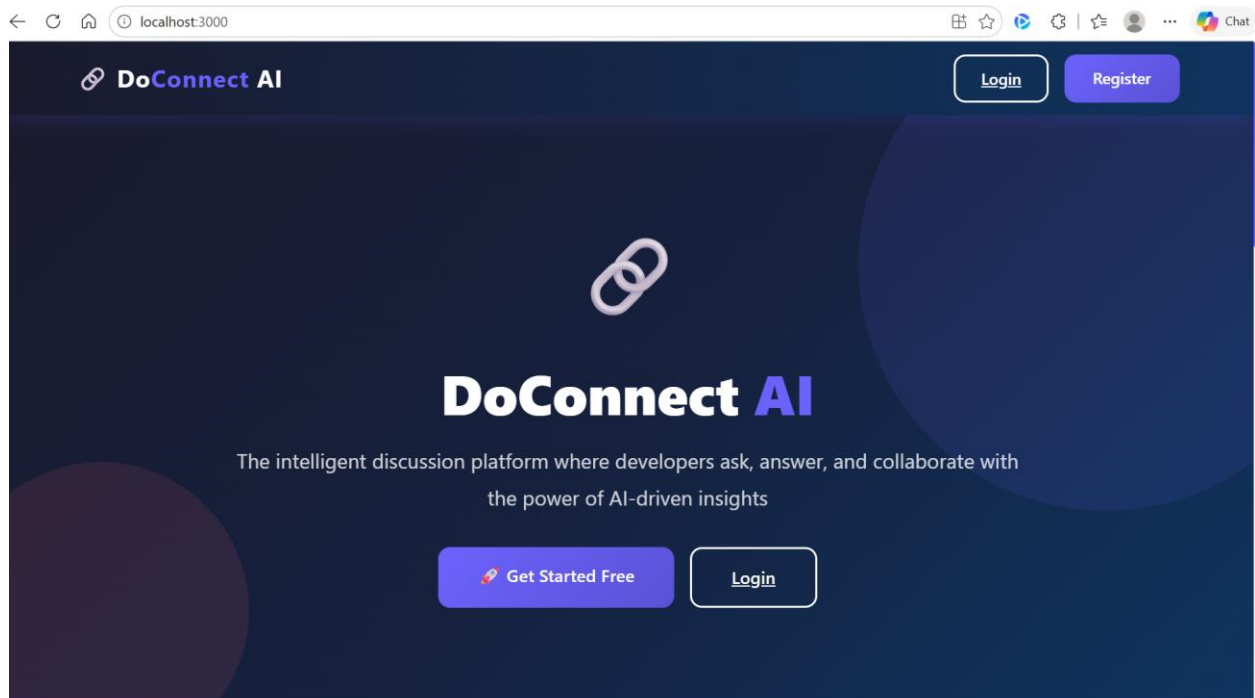




11. Future Enhancements

- Push notifications and email alerts for new answers and chat mentions
- Advanced full-text search with filtering and ranking
- AI-based automatic question categorization and duplicate detection
- File attachment and image upload support for questions and answers
- Caching layer (Redis) for high-traffic trending content
- Mobile application using React Native sharing the existing backend

12. UI OUTPUTS



The image displays two screenshots of the DoConnect AI web application interface. Both screenshots show a dark blue header with the DoConnect AI logo and navigation links for Login and Register.

Top Screenshot (localhost:3000/login):

- Left Panel:** Features the DoConnect AI logo, a welcome message "Welcome back to DoConnect AI", and a login form with fields for Email Address (you@example.com) and Password (masked with dots). A "Sign In" button is present, along with a link to "Create one free" for new users.
- Right Panel:** A hero section with a robot illustration, a quote: "The AI suggestions are incredibly accurate. It saves me hours of research!" attributed to Priya M., Full Stack Dev. It also displays statistics: 10K+ Users, 50K+ Questions, and 100K+ Answers.

Bottom Screenshot (localhost:3000/register):

- Left Panel:** Features the DoConnect AI logo, a message "Join thousands of developers", and a registration form with fields for Username (cooldev123), Email (you@example.com), and Password (Min 6 chars). It includes a "Register As" dropdown menu set to "Regular User" and a "Create Account" button.
- Right Panel:** A hero section with a speech bubble illustration, a quote: "Real-time chat makes collaboration so much faster and easier." attributed to Arjun K., Software Engineer. It also displays the same statistics: 10K+ Users, 50K+ Questions, and 100K+ Answers.

← ↻ 🏠 localhost:3000/profile ⚙️ ☆ 🔄 ⚙️ | 👤 ... 🗨️ Chat

DoConnect AI 🏠 Home ? Ask 💬 Chat 👤 Poojitha Logout

👤 Profile 🔒 Password ? Questions (2) 💡 Answers (2)

Edit Profile

Username

Poojitha

Bio

Im a fullstack developer

Save Changes ✓

← ↻ 🏠 localhost:3000/profile ⚙️ ☆ 🔄 ⚙️ | 👤 ... 🗨️ Chat

DoConnect AI 🏠 Home ? Ask 💬 Chat 👤 Poojitha Logout

**Poojitha**
poojitha@gmail.com
USER

**2**
Questions

**2**
Answers

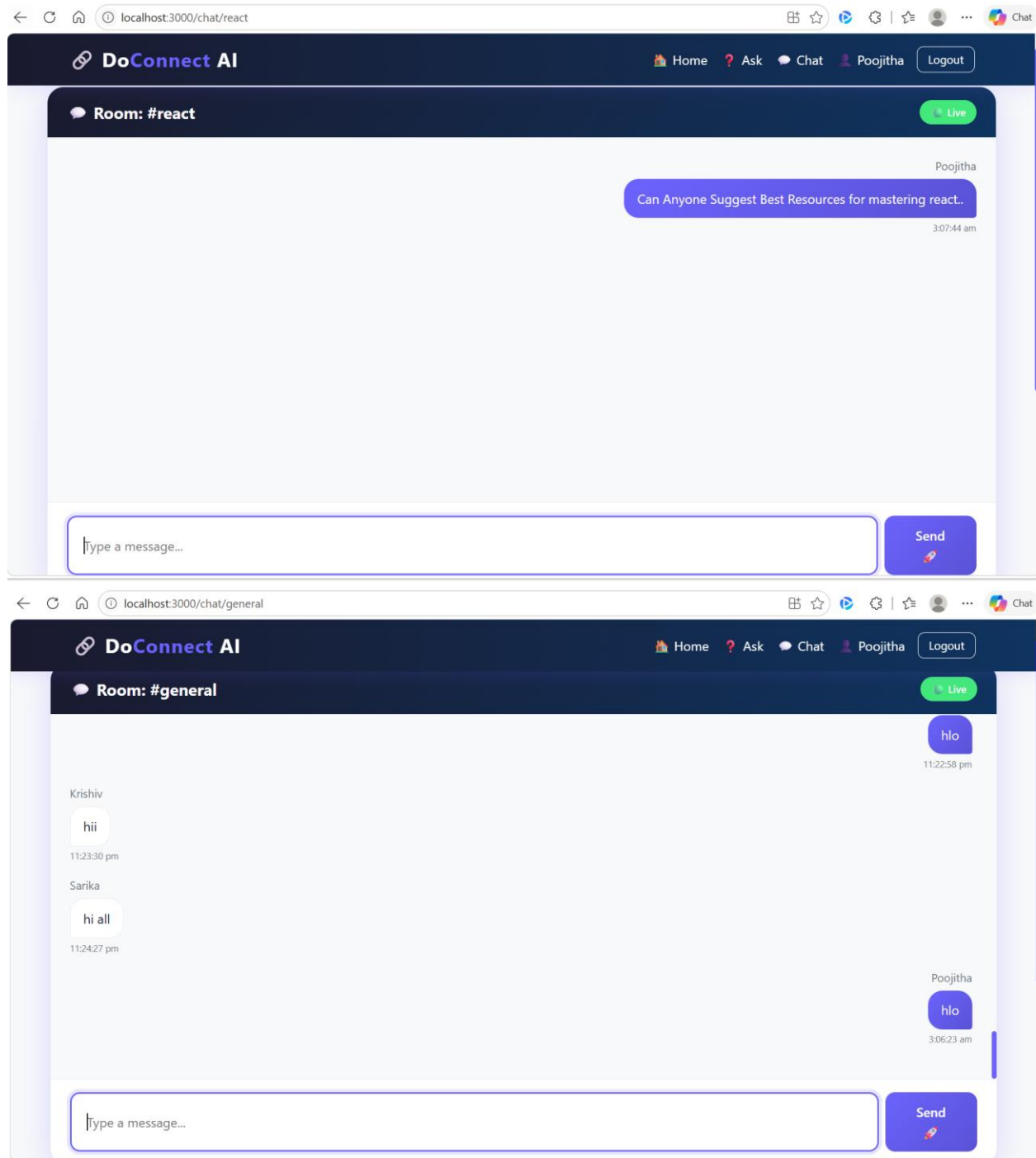
**15/6/2026**
Member Since

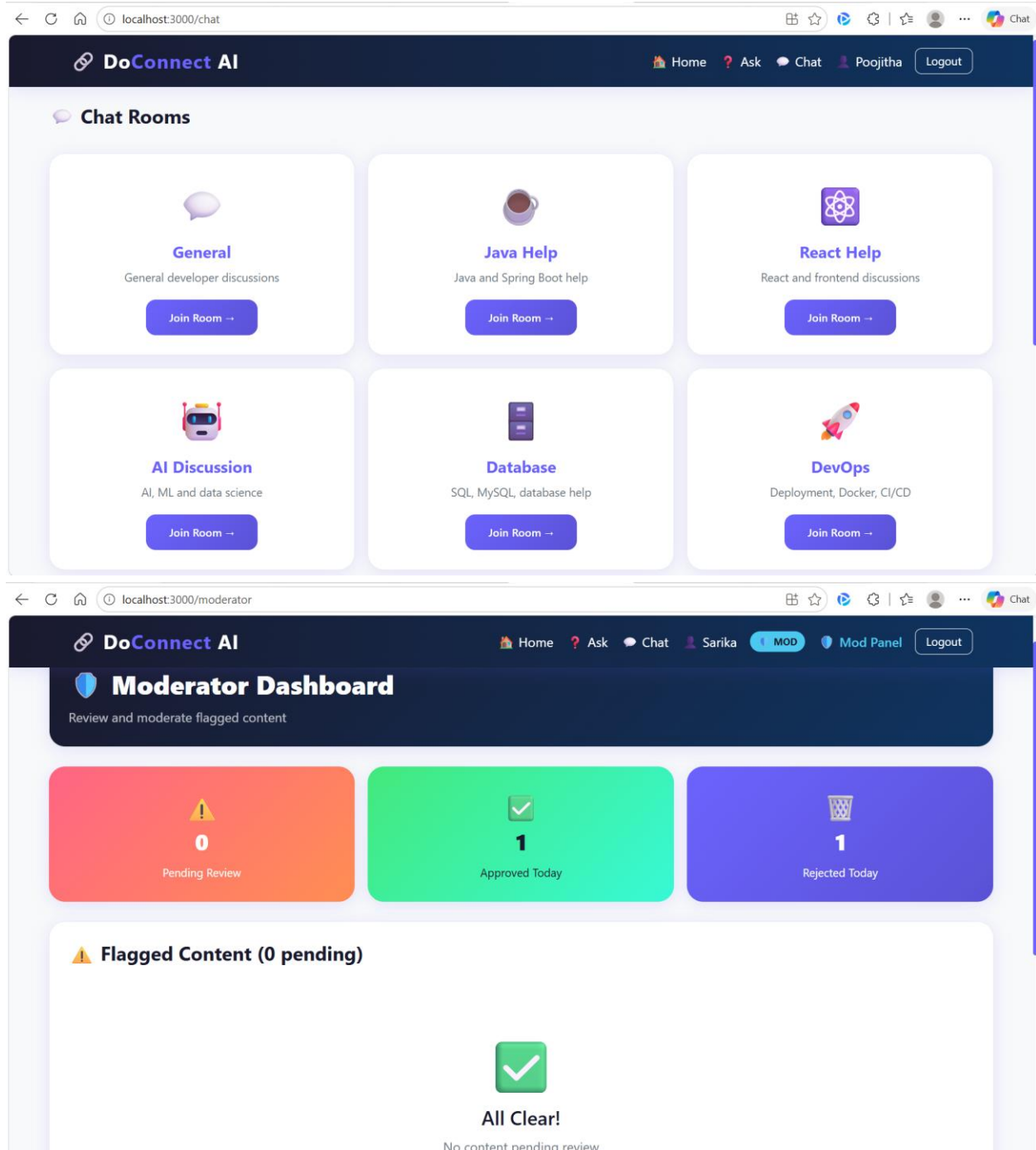
👤 Profile 🔒 Password ? Questions (2) 💡 Answers (2)

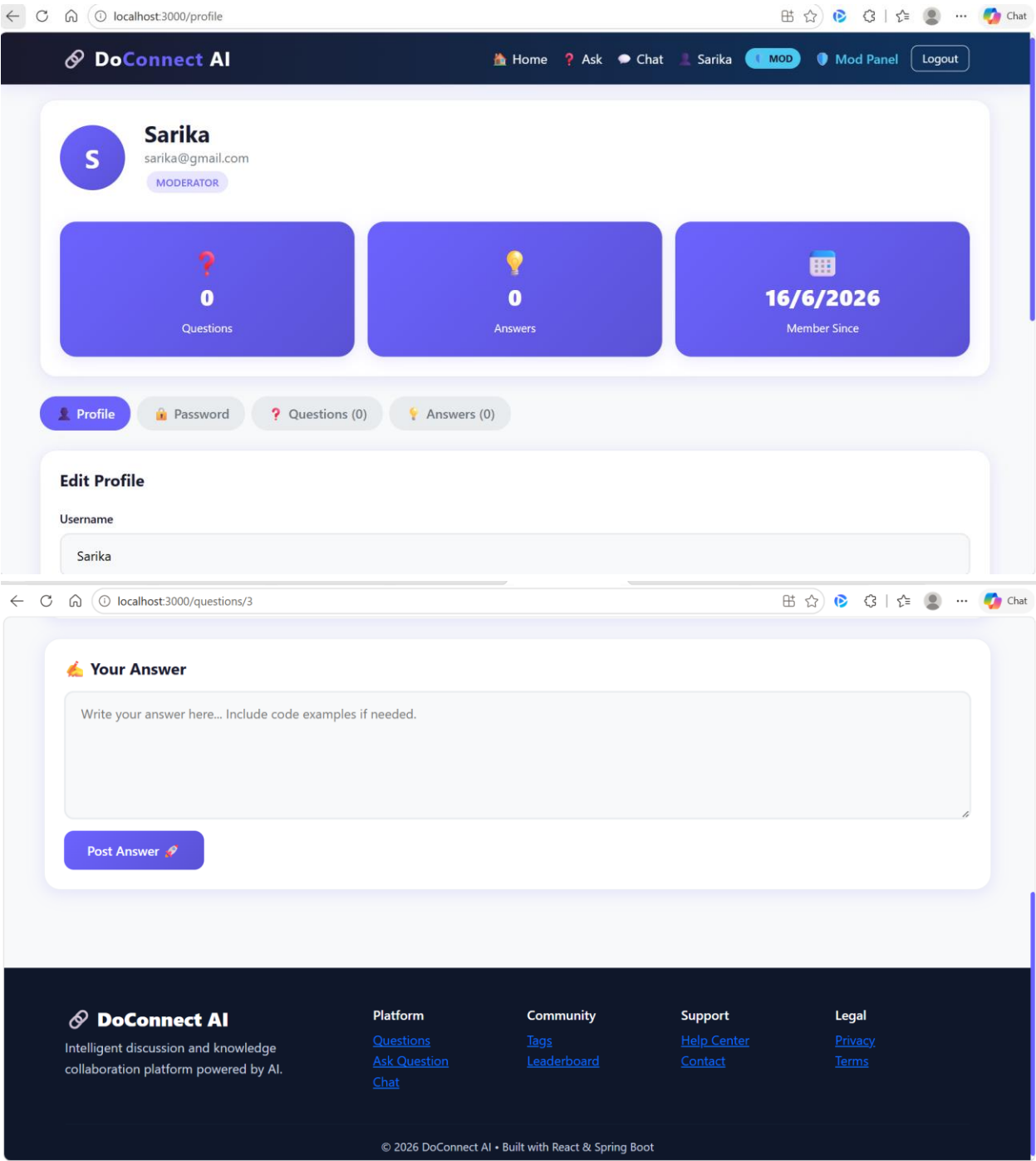
Edit Profile

Username

Poojitha







The image displays two screenshots of the DoConnect AI web application. The top screenshot shows a question page for 'Why is my Spring Boot microservice not registering with Eureka?'. The question is asked by Divya and has 1 view. It includes tags for 'springboot', 'microservices', 'eureka', and 'eureka server'. There is a 'Report' button and a 'Generate AI Answer' button. The bottom screenshot shows the profile of a user named Krishiv, who is an admin. The profile includes a circular avatar with the letter 'K', the email 'krishiv@gmail.com', and a status of 'ADMIN'. Below the profile information are three statistics: 1 Question, 1 Answer, and 17/6/2026 Member Since. At the bottom, there is an 'Edit Profile' section with a 'Username' field containing the name 'Krishiv'.

DoConnect AI Home ? Ask Chat Sarika MOD Mod Panel Logout

Why is my Spring Boot microservice not registering with Eureka?

My service is running but not appearing in Eureka dashboard. I have configured application.properties. What could be wrong?

springboot microservices eureka eureka server

Asked by Divya · 1 views

0 Report

AI Answer Assistant Generate AI Answer

Click the button to get an AI-powered answer using Gemini

1 Answers

AI service unavailable.

0

By Divya

DoConnect AI Home ? Ask Chat Krishiv ADMIN Admin Panel Logout

Krishiv
krishiv@gmail.com
ADMIN

1
Questions

1
Answers

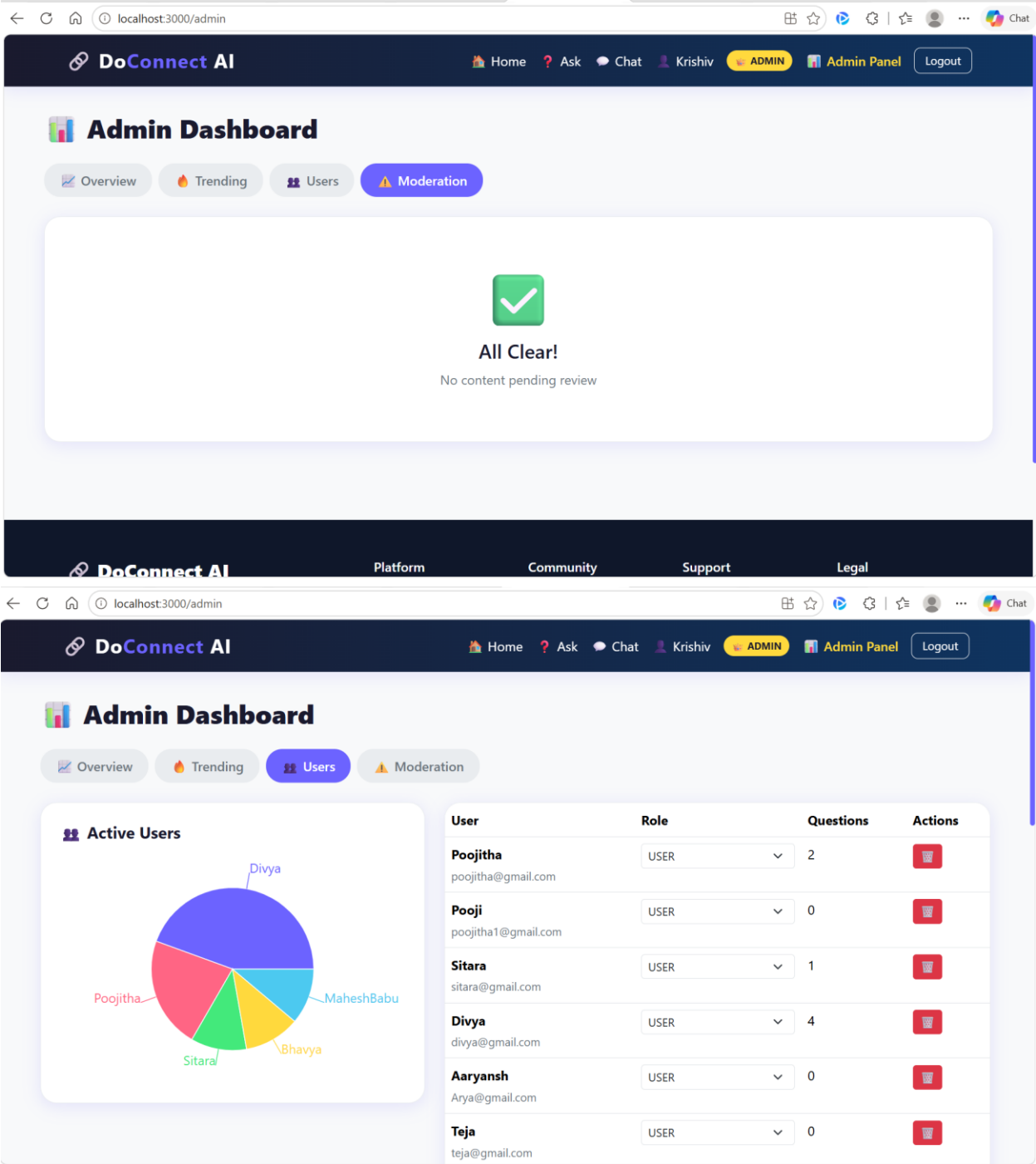
17/6/2026
Member Since

Profile Password Questions (1) Answers (1)

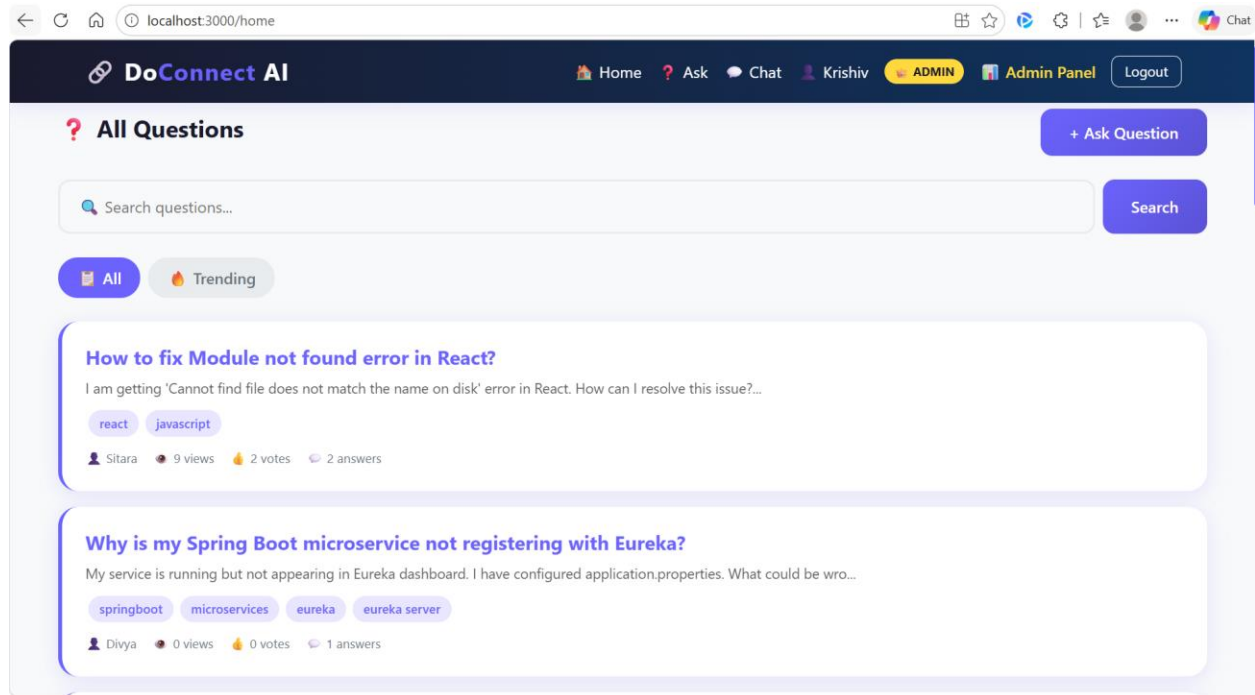
Edit Profile

Username

Krishiv







12. Conclusion

DoConnect AI successfully demonstrates the design and implementation of a complete, enterprise-style full-stack application. It integrates secure JWT-based authentication, a normalized relational database, real-time WebSocket communication, generative AI features powered by Google Gemini, and a clean, pattern-driven backend architecture.

The project covers the full software development lifecycle — from requirement analysis and system design through implementation, unit testing, documentation, and deployment — and reflects practical, production-relevant engineering practices suitable for a modern AI-integrated web platform.
